

Hotness- and Lifetime-Aware Data Placement and Migration for High-Performance Deep Learning on Heterogeneous Memory Systems

Myeonggyun Han[✉], Jihoon Hyun, Seongbeom Park[✉], and Woongki Baek[✉], *Member, IEEE*

Abstract—Heterogeneous memory systems that comprise memory nodes with disparate architectural characteristics (e.g., DRAM and high-bandwidth memory (HBM)) have surfaced as a promising solution in a variety of computing domains ranging from embedded to high-performance computing. Since deep learning (DL) is one of the most widely-used workloads in various computing domains, it is crucial to explore efficient memory management techniques for DL applications that execute on heterogeneous memory systems. Despite extensive prior works on system software and architectural support for efficient DL, it still remains unexplored to investigate heterogeneity-aware memory management techniques for high-performance DL on heterogeneous memory systems. To bridge this gap, we analyze the characteristics of representative DL workloads on a real heterogeneous memory system. Guided by the characterization results, we propose HALO, hotness- and lifetime-aware data placement and migration for high-performance DL on heterogeneous memory systems. Through quantitative evaluation, we demonstrate the effectiveness of HALO in that it significantly outperforms various memory management policies (e.g., 28.2 percent higher performance than the HBM-Preferred policy) supported by the underlying system software and hardware, achieves the performance comparable to the ideal case with infinite HBM, incurs small performance overheads, and delivers high performance across a wide range of application working-set sizes.

Index Terms—Hotness- and lifetime-aware data placement and migration, high-performance deep learning, heterogeneous memory systems

1 INTRODUCTION

HETEROGENEOUS memory systems that consist of memory nodes with a wide range of architectural characteristics are rapidly emerging as a promising solution in various computing domains ranging from embedded [15] to high-performance computing [23]. The key idea behind the heterogeneous memory systems is to provide useful properties such as performance, energy efficiency, durability, and cost efficiency that cannot be achieved by solely employing a single type of memory.

For instance, the recently-released Intel Xeon Phi KNL architecture consists of high-bandwidth memory (HBM) and DRAM nodes that are optimized for high bandwidth (i.e., HBM) and large capacity (i.e., DRAM), respectively. It is the responsibility of the underlying system software and/or hardware to effectively manage the heterogeneous memory nodes in the underlying system to maximize the metric of interest such as performance and energy efficiency.

Since deep learning (DL) is one of the representative workloads in a variety of computing domains, it is crucial to investigate efficient system software support to optimize the performance of DL on heterogeneous memory systems. In particular, as a large number of widely-used DL algorithms

employ advanced network architectures where tensors exhibit significantly different characteristics [40], the system software support that robustly performs optimizations considering the characteristics of the tensors and heterogeneous memory nodes is important for high-performance DL. Despite the extensive prior works [7], [13], [21], [28], [40], [47] on the system software and architectural support for high-performance DL, it still remains unexplored to investigate the design and implementation of the effective memory management techniques to improve the performance of DL on heterogeneous memory systems.

To bridge this gap, we propose HALO, hotness- and lifetime-aware data placement and migration for high-performance DL on heterogeneous memory systems. HALO extracts the hotness and lifetime information on the tensors of the target DL application based on its dataflow graph.¹ HALO dynamically places and migrates the tensors on heterogeneous memory nodes based on their hotness and lifetime characteristics. We implement HALO based on the widely-used TensorFlow system [7] and demonstrate that it significantly improves the performance and energy efficiency of various DL applications on a real heterogeneous memory system without any hardware modifications.

Specifically, this paper makes the following contributions:

- We present an in-depth characterization of widely-used DL applications in terms of the execution time

1. Intuitively, the hotness and lifetime of a tensor represent the number of operations that access the tensor and the difference between its deallocation and allocation times (see Section 2.4).

• The authors are with the School of Electrical and Computer Engineering, Ulsan National Institute of Science and Technology (UNIST), 44919 Ulsan, Republic of Korea. E-mail: {hmg0228, jhyun0812, amita90, wbaek}@unist.ac.kr.

Manuscript received 16 Feb. 2019; revised 9 Oct. 2019; accepted 20 Oct. 2019. Date of publication 25 Oct. 2019; date of current version 7 Feb. 2020.

(Corresponding author: Woongki Baek.)

Recommended for acceptance by B. Parhami.

Digital Object Identifier no. 10.1109/TC.2019.2949408

and tensor characteristics on a full heterogeneous memory system. Our characterization results show that tensors of the evaluated DL applications exhibit disparate characteristics in terms of the capacity, hotness, and lifetime.

- Guided by the characterization results, we propose HALO, hotness- and lifetime-aware data placement and migration for high-performance DL on heterogeneous memory systems. HALO analyzes the hotness and lifetime characteristics of the tensors of the target DL application without requiring any offline profiling. We formulate the tensor placement and migration problem as a variant of the NP-hard knapsack problem with time intervals [10] and propose an efficient algorithm to address it. HALO dynamically places and migrates the tensors across the heterogeneous memory nodes in a hotness- and lifetime-aware manner based on the proposed algorithm.
- We present the design and implementation of HALO by extending TensorFlow [7], which is one of the most widely-used machine-learning systems. We demonstrate that the proposed techniques can be effectively implemented on top of the production-quality machine-learning system.
- We quantify the effectiveness of HALO using a full heterogeneous memory system and representative DL applications including the state-of-the-art convolutional neural networks (CNNs), a recurrent neural network (RNN), and a network with an attention mechanism [1], [2], [3], [5], [25], [34], [37], [38], [46]. Our quantitative evaluation demonstrates the effectiveness of HALO in that it significantly outperforms various memory management policies (e.g., 28.2 percent higher performance than the HBM-Preferred policy) supported by the underlying system software and hardware, delivers the performance comparable to that of the ideal case with infinite HBM, incurs small performance overheads, and achieves high performance across various application working-set sizes. To the best of our knowledge, our work is the first to propose, implement, and evaluate the hotness- and lifetime-aware memory management technique for high-performance DL on heterogeneous memory systems.

The rest of this paper is organized as follows. Section 2 provides the background information for this work. Section 3 describes the experimental methodology. Section 4 investigates the characteristics of deep-learning applications on a full heterogeneous memory system. Section 5 presents the design and implementation of HALO. Section 6 quantifies the effectiveness of HALO. Section 7 summarizes related work. Section 8 concludes the paper.

2 BACKGROUND

2.1 TensorFlow Machine-Learning System

TensorFlow is a widely-used machine-learning (ML) system [7]. It allows for programmers to express their ML algorithms as dataflow graphs. A *dataflow graph* mainly consists of tensors and operations. *Tensors* are multidimensional arrays, whose elements have one of the basic primitive data

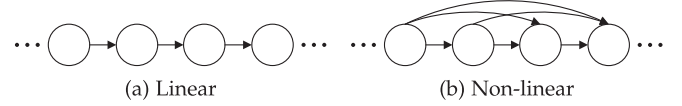


Fig. 1. Networks with linear and non-linear connections.

types such as float32. An *operation* takes zero or more input tensors and produces zero or more output tensors [7].

The dataflow graph of the target TensorFlow application is a directed acyclic graph, where the nodes and edges represent the operations and tensors. The source (or destination) node of an edge represents the producer (or consumer) of the corresponding tensor. The nodes in ML applications are typically executed in a sequential manner as the layers in their network architectures are sequentially connected.

The dataflow graph of the target TensorFlow application remains unchanged through the execution of the application. Therefore, static analysis and optimization techniques based on the dataflow graph of the target TensorFlow application are widely-used and effective. As discussed in Section 5, HALO also builds on static analysis and optimization techniques.

A recent work classifies network architecture types into *linear* and *non-linear* network architectures [40]. Fig. 1 shows the network architectures with linear and non-linear architectures. With the linear network architecture, edges exist only between consecutive nodes in the sequential execution order of the operations. In contrast, with the non-linear network architecture, edges may exist between non-consecutive nodes in the sequential execution order. This indicates that the variance in the hotness and lifetime of the tensors tends to be larger with the non-linear network architecture as some tensors can be consumed by more operations than the others.

We focus on the efficiency optimization of the training phase of deep-learning (DL) applications. In each *training epoch* (or epoch), a DL application iterates all of the training data to train its model. In each *training step* (or step), the DL application processes a batch of the training data. For example, if 70,000 images are used as the training data with a batch size of 100, an epoch consists of 700 steps.

We implement and evaluate HALO by extending TensorFlow owing to its widespread use and well-established development environment. However, we believe that the design of HALO is sufficiently generic to be applicable to other ML platforms such as Caffe [21] and PyTorch [29].

2.2 NUMA-Aware Memory Policies

Non-uniform memory access (NUMA) systems comprise multiple *processor nodes*, each of which consists of a CPU and local memory with one or more DIMMs. Processor nodes are connected via the interconnection network (ICN). Modern NUMA systems typically support cache coherence across the processor nodes to provide an abstraction of a single globally addressable memory space.

Accesses to a memory object on a NUMA system are largely classified into two categories – *local* and *remote* accesses. A local (or remote) access is performed when a processor node accesses a memory object that is located on the same (or different) processor node. Remote accesses incur a longer latency than that of local accesses because they generate packets that are transmitted over the ICN. Thus, frequent

remote accesses may significantly degrade the overall performance of NUMA systems [17].

The Linux kernel provides several NUMA-aware memory placement policies including the following three representative policies – the *local*, *interleave*, and *preferred* policies. The local policy places a page in the local memory of the processor node where the task that first touches the page is running. The interleave policy places pages across the processor nodes in a round-robin manner without considering the physical location of the tasks that access the pages. The preferred policy first attempts to place pages in the memory of user-specified processor nodes. If there is no sufficient space in the user-specified nodes, it places pages in other nodes. The local policy generally achieves higher locality, whereas the interleave policy generally achieves better load balancing across the nodes.

2.3 Heterogeneous Memory Systems

Heterogeneous memory systems comprise two or more types of memory nodes (e.g., non-volatile memory (NVM) and DRAM [16], high-bandwidth memory (HBM) and DRAM [35]) that exhibit widely-different characteristics such as bandwidth, latency, persistence, and durability.

Heterogeneous memory systems can be largely classified into two categories. The heterogeneous memory systems that belong to the first category organize heterogeneous memory nodes in a *flat* manner in that the heterogeneous memory nodes are mapped to the single physical address space [9], [22], [33], [39], [41], [42], [44]. The heterogeneous memory systems in the second category organize heterogeneous memory nodes in a *tiered* manner in that faster and smaller memory nodes are used as hardware caches for slower and larger memory nodes [14], [18], [43], [45]. None of the proposed systems in both categories use the application-level information available in the dataflow graph of the target DL application, achieving suboptimal performance. In contrast, HALO significantly improves the performance of the target DL application on heterogeneous memory systems with the flat memory organization by effectively utilizing the application-level information (e.g., hotness, lifetime).

We assume that the underlying system is similar to the Intel Xeon Phi KNL. This is done because it represents advanced and generic heterogeneous memory systems² in which the computation units can directly and simultaneously access the data from each of the heterogeneous memory nodes and the access to the full software stack (e.g., OS, DL framework) is publicly available.

A recent work has characterized the performance of the HBM and LBM nodes³ using a Xeon Phi KNL CPU-based system [30]. Since HBM employs a high-performance multi-channel interface between the HBM nodes and the CPU, it achieves significantly higher bandwidth (i.e., 480 GB/s versus 80 GB/s) than LBM. In contrast, the HBM nodes incur

slightly longer latency (i.e., 160-175 nanoseconds versus 130-140 nanoseconds) than the LBM nodes [30].

Further, the capacity of HBM is significantly smaller (i.e., 16 GB versus 192 GB) than LBM as the area budget of HBM is tightly limited to employ the high-bandwidth package-on-package technology. Since DL applications typically require a large amount of physical memory, their working-set size often significantly exceed the capacity of HBM.

The Xeon Phi KNL architecture provides the Flat mode (i.e., flat memory organization) in which the HBM and LBM nodes are mapped to a single physical address space. Among the various Flat modes supported by the Xeon Phi KNL architecture, we employ the SNC4 mode because it consistently achieves high performance across a wide range of applications [30]. In the SNC4 mode, the Xeon Phi KNL architecture exposes four HBM nodes and four LBM nodes to the operating system (OS). The OS manages the HBM and LBM nodes as eight NUMA nodes.

In addition, the Xeon Phi KNL architecture provides the Cache mode (i.e., tiered memory organization). In the Cache mode, HBM is fully managed by hardware as a last-level hardware cache and transparent to the OS. In this work, along with the other representative software-based memory management policies, we compare the performance of HALO with the hardware-based memory management policy based on the Cache mode.

2.4 Terminology

In this section, we provide the definitions of the two key concepts (i.e., tensor hotness and lifetime). The hotness of a tensor is defined as the sum of its consumer count and one (i.e., the producer count). Intuitively, a tensor with a larger hotness value is expected to generate a larger amount of data transfers (per byte).

To define the lifetime of a tensor, we consider the linearized execution order of the operations in the target DL application and assume that each instruction is assigned an index in the execution order. The allocation (i.e., start) time of a tensor is defined as the index of the producer. The deallocation (i.e., end) time of a tensor is defined as the index of the last consumer (among all the consumers) in the linearized execution order. The lifetime of a tensor is then defined as the difference between the deallocation and allocation times.

3 EXPERIMENTAL METHODOLOGY

3.1 System Configuration

To quantify the effectiveness of HALO and various software- and hardware-based memory management policies, we employ a 64-core server system equipped with a single Xeon Phi KNL many-core CPU [35]. Table 1 summarizes the configuration of the evaluated server system. To ensure the high performance of mathematical primitives that are frequently executed by deep-learning (DL) applications, we employ the Intel Math Kernel Library (MKL), which is highly optimized for the Xeon Phi KNL architecture.

3.2 Deep-Learning Applications

Table 2 summarizes the evaluated DL applications. Specifically, we use nine widely-used DL applications including

2. Currently-available GPUs can be considered as a restricted type of heterogeneous memory systems in that the computation units in GPUs can only access the data from the device memory and the data must be always copied from the host memory to the device memory before it can be accessed by the computation units in GPUs.

3. We denote multi-channel DRAM (MCDRAM) and DRAM as high-bandwidth memory (HBM) and low-bandwidth memory (LBM).

TABLE 1
System Specification

Component	Description
Processors	Intel Xeon Phi Processor 7230 @ 1.3 GHz, 64 cores per CPU
L1 I-cache	Private, 32 KB, 8 ways
L1 D-cache	Private, 32 KB, 8 ways
L2 cache	Private (per tile), 1 MB, 16 ways
Memory	192 GB DDR4 (LBM) + 16 GB MCDRAM (HBM)
TensorFlow	1.4.0
System software	Intel MKL 0.9, GCC 5.5.0, Intel Python 2.7.4, CentOS 7, Linux Kernel 3.10.0

state-of-the-art CNNs, an RNN, and a network with an attention mechanism – AN, DN, GN, INv4, LSTM, RN, RNv2, TR, and VGG [1], [2], [3], [5], [25], [34], [37], [38], [46]. They exhibit widely-different characteristics in terms of the network linearity, the operation count, and the tensor count.

For each application, we set the parallelism parameter of TensorFlow to 64, which determines the number of threads that execute each operation in a parallel manner. This is done because the use of this value consistently provides the highest performance across the applications. Further, we set the Intel MKL parallelism parameter, which controls the number of threads to execute the mathematical primitive operations implemented in the Intel MKL, to 64 for a similar reason. Our parameter settings are in line with earlier findings [6].

Table 2 summarizes the batch sizes used for each application. In general, as the batch size increases, the working-set size of the application also increases. We use three batch sizes (i.e., small, medium, and large) for the applications. For the performance characterization studies, we use the small batch size to investigate the performance sensitivity of the applications when most of the data is allocated to LBM or HBM. To quantify the effectiveness of HALO and the other memory policies, we use all the three batch sizes.

We collect and analyze various performance and energy consumption data for each DL application through the performance event counters provided by TensorFlow, Linux, and the Performance Monitoring Counters (PMCs) available in the Intel Xeon Phi KNL architecture.

4 CHARACTERIZATION OF DL APPLICATIONS

This section investigates the characteristics of the deep-learning (DL) applications on the evaluated heterogeneous memory system in terms of the execution time and tensor characteristics. We present more detailed data with VGG and GN, which represent DL applications that employ the linear and non-linear networks, respectively. While omitted for conciseness, the other DL applications exhibit the data trends similar to those of the aforementioned applications.

4.1 Execution Time Characteristics

We first analyze the per-operation execution time of the DL applications. Figs. 2a and 2b show the per-operation execution time of VGG when all of the tensors are allocated to LBM and HBM, respectively.⁴ In addition, Fig. 2c shows the

per-operation speedup of HBM over LBM when executing VGG. We observe the following data trends.

First, a small number of operation types account for a majority of the total execution time, which is in line with the findings from a recent work [8]. To gain a deeper insight into this data trend, Fig. 3 shows a breakdown of the execution time of each application, normalized to the execution time with all the tensors allocated to LBM. We observe that the operations such as the convolution (Conv), MaxPool (MaxPool), and rectifier (Relu) operations account for a significant portion of the total execution time.

Second, a few operation types (e.g., FusedBatchNorm, Relu, MaxPool) achieve higher speedup with HBM than the other operation types. Since such operations perform relatively simple computations compared with the other operations, the memory access time for such operations accounts for a larger portion of the total execution time than the other operations, making their performance highly sensitive to the memory bandwidth.

Third, the execution time data across the operations exhibits symmetric trends from the middle (Figs. 2a and 2b). This mainly occurs because the first half of the operations performed during the forward path of the training and the second half of the operations are performed during the backpropagation path, which is essentially the inverse of the forward path.

Fourth, the execution time of each operation type tends to decrease towards the end of the forward path (or the beginning of the backpropagation path). This is mainly because the size of the tensors is reduced (or increased) as they are processed by operations such as MaxPool in the forward path (or the backpropagation path). This indicates that operations near the beginning of the forward path and the end of the backpropagation path account for a large portion of the total execution time.

Fig. 4 shows the per-operation execution time of GN, which employs a non-linear network. We observe that GN exhibits the per-operation execution time characteristics similar to those of VGG. In addition, while omitted for conciseness, the other DL applications evaluated in this work exhibit similar characteristics.

4.2 Tensor Characteristics

We investigate the characteristics of tensors of the DL applications in terms of their size, hotness, and lifetime. Fig. 5 shows the tensor characteristics of VGG.

First, we observe that tensors exhibit widely-different hotness characteristics. Some tensors are only consumed by the next operation in the sequential execution order of operations. In contrast, other tensors produced by operations in the forward path are consumed not only by the next operation in the execution order but also by the associated operations in the backpropagation path.

Second, tensors also exhibit widely-different lifetime characteristics due to the aforementioned reasons. These observations indicate that data placement and migration techniques for high-performance DL on heterogeneous memory systems must judiciously consider the hotness and lifetime characteristics of each tensor to improve the overall performance.

Third, similarly to the per-operation execution time results, the tensor size data shows symmetric trends from the middle

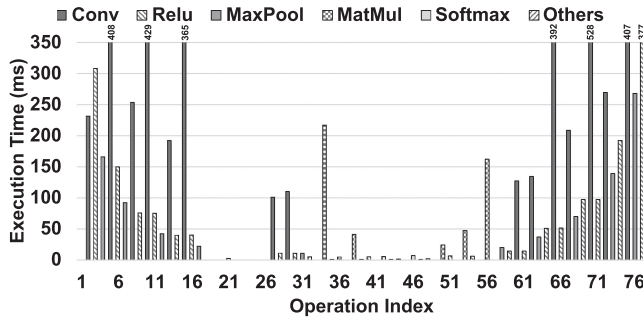
⁴ In Fig. 2 (and also in Fig. 4), the top five time-consuming operations are highlighted with different colors and patterns.

TABLE 2
Evaluated Deep-Learning Applications

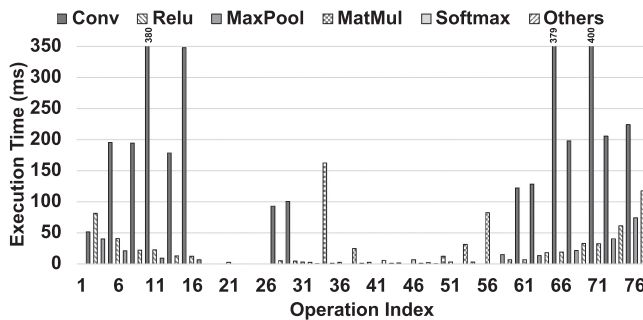
Application	Dataset	Network	Depth	N_{Ops}	$N_{Tensors}$	Batch size (S, M, L)
AlexNet (AN)	Synthetic [1]	Linear	8	61	112	2950, 11000, N/A
DenseNet (DN)	CIFAR-10 [24]	Non-linear	40	2075	2783	2450, 4000, 8000
GoogLeNet (GN)	Synthetic [1]	Non-linear	22	366	877	1160, 1800, 2650
Inception-v4 (INv4)	ImageNet [32]	Non-linear	75	5185	7750	185, 300, 600
RNN with LSTM (LSTM)	PTB [27]	Non-linear	(2, 2400)	3672	2655	150, 4200, 6100
ResNet (RN)	ImageNet [32]	Non-linear	50	2210	3158	380, 600, 1000
ResNet-v2 (RNv2)	ImageNet [32]	Non-linear	50	2019	2830	440, 700, 1000
Transformer (TR)	WMT 2014 ENDE [4]	Non-linear	28	17505	15904	4000, 26000, 47000
VGG (VGG)	Synthetic [1]	Linear	11	77	160	205, 720, 820

(Fig. 5a). This is mainly because the same set of operations in the forward path are performed in a reverse order in the back-propagation path.

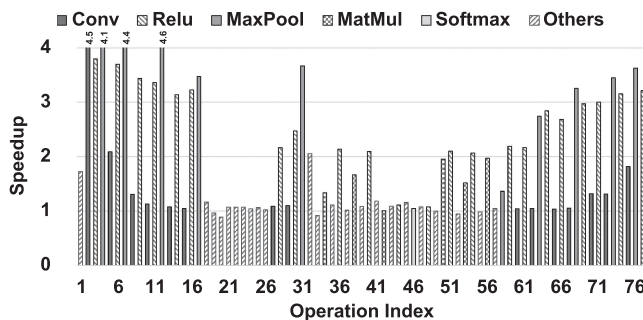
Fourth, the size of the tensors tends to decrease (or increase) as they are allocated near the end (or beginning) of the forward (or backpropagation) path. This mainly arises because the size of tensors is significantly reduced (or increased) as they are processed by operations such as Max-Pool in the forward path (or the backpropagation path).



(a) LBM



(b) HBM



(c) Speedup

Fig. 2. Per-operation execution time of VGG.

Fig. 6 shows the tensor characteristics of GN, which employs a non-linear network. We observe that GN exhibits the tensor characteristics similar to those of VGG. One notable difference is that GN shows larger variations in terms of the tensor hotness and lifetime than VGG. GN employs a convolutional neural network with non-linear connections in which some tensors that are allocated in a path (i.e., forward or backpropagation) are consumed by multiple operations in the same path, making their hotness and lifetime characteristics significantly different from those of the other tensors. This observation indicates that hotness- and lifetime-aware memory management becomes even more crucial for high-performance DL with more advanced neural network architectures on heterogeneous memory systems.

4.3 Summary

We summarize the characterization results of the DL applications as follows.

- Tensors exhibit a wide range of characteristics in terms of the hotness and lifetime. Since HBM is a highly limited resource in heterogeneous memory systems, it is crucial to place and migrate the tensors across the heterogeneous memory nodes in a hotness- and lifetime-aware manner in order to maximize the performance of the target DL application.
- Operations also exhibit widely-different characteristics in terms of the execution time and performance sensitivity to the memory bandwidth. Since data-intensive operations are highly sensitive to the memory bandwidth, it is crucial to identify the tensors associated with the data-intensive operations based on their hotness and lifetime characteristics and efficiently place and migrate these tensors across the

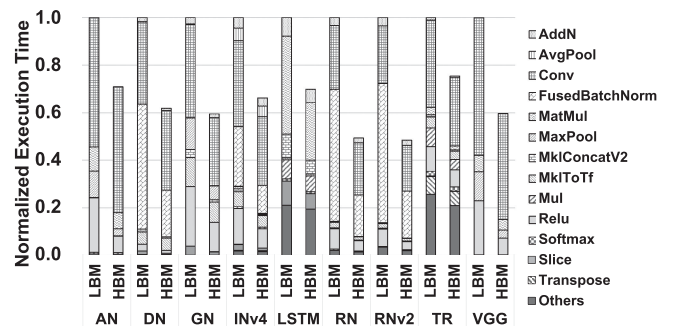


Fig. 3. Execution time breakdowns.

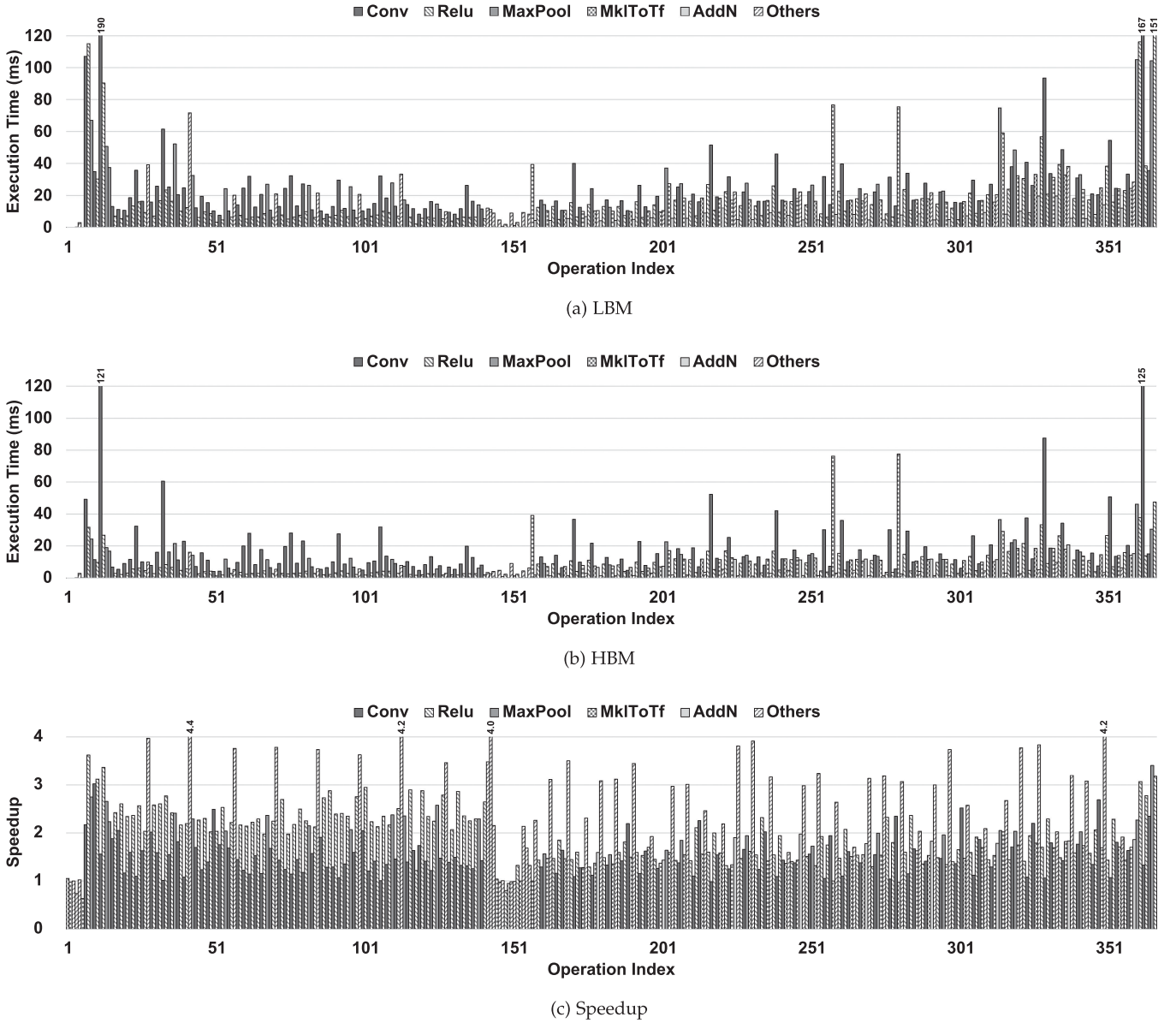


Fig. 4. Per-operation execution time of GN.

heterogeneous memory nodes in order to improve the overall performance of the target DL application.

5 DESIGN AND IMPLEMENTATION

HALO is a software-based system that dynamically places and migrates tensors across the heterogeneous memory nodes based on their characteristics in order to improve the performance of the target deep-learning (DL) application. As shown in Fig. 7, HALO mainly comprises the following four components – (1) the tensor hotness analyzer, (2) the tensor lifetime analyzer, (3) the tensor combiner, and (4) the tensor manager.

5.1 Tensor Hotness Analyzer

The tensor hotness analyzer estimates the access frequency (i.e., hotness) of each tensor used by the target DL application. Section 2.4 provides the definition of the tensor hotness.

Before executing the target DL application, TensorFlow constructs an initial dataflow graph, performs a sequence of optimizations, and generates an optimized dataflow graph. We have extended the metadata for the tensor to encode the consumer count. In addition, we have added an additional dataflow graph optimization pass to compute the hotness of each tensor.

5.2 Tensor Lifetime Analyzer

The tensor lifetime analyzer estimates the lifetime of each tensor used by the target DL application. TensorFlow dynamically tracks the usage of each tensor using a reference counting technique. When the reference counter of a tensor becomes zero, TensorFlow deallocates the memory used by the tensor.

To estimate the lifetime of each tensor, the tensor lifetime analyzer first generates the linearized execution order of the operations of the target DL application, which is equivalent to the actual linearized execution order produced by the

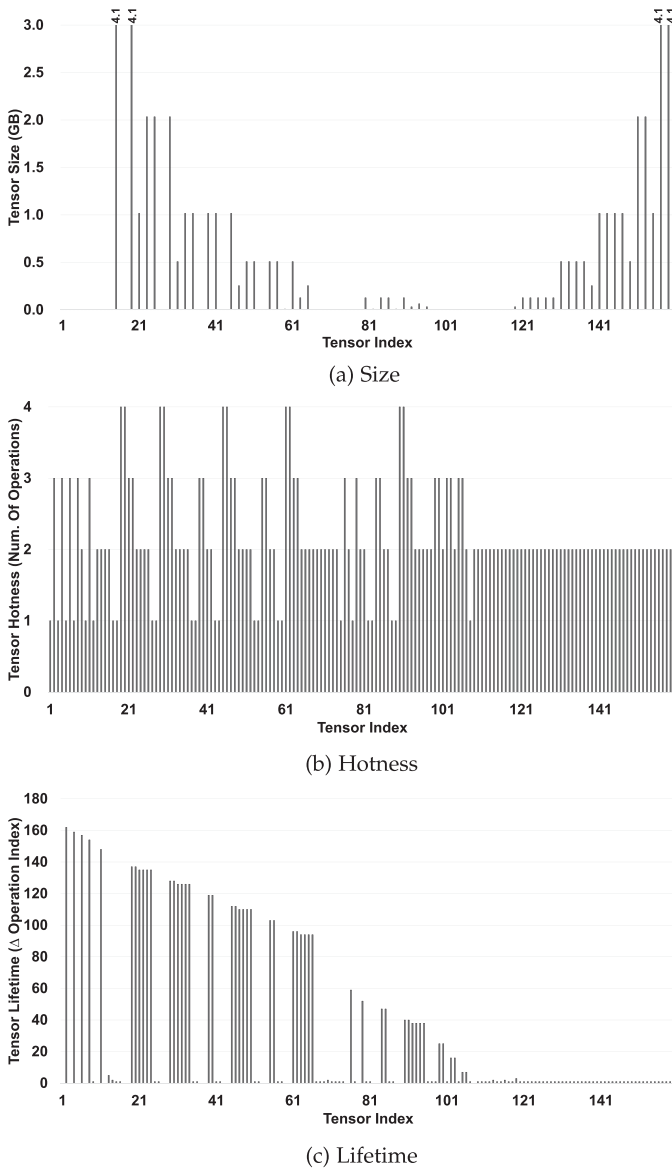


Fig. 5. Tensor characteristics of VGG.

TensorFlow scheduler. The tensor lifetime analyzer assigns each operation a unique integer value (i.e., index) in the linearized execution order of the operations. For instance, if the index of an operation is 3, it is the third operation in the linearized execution order. The tensor lifetime analyzer then computes the lifetime of each tensor based on its definition provided in Section 2.4.

5.3 Tensor Combiner

TensorFlow performs an optimization that reuses the memory buffer of an input tensor as the memory buffer of an output tensor for predefined operations such as `MklInputConversion` and `Mkl2Tf`. This optimization is performed to eliminate the overheads of deallocating the memory buffer allocated for an input tensor and allocating the new memory buffer for an output tensor if the dimensions of the two tensors are identical.

Since tensors affected by this optimization share the same memory buffer, they essentially behave as a single (virtually) combined tensor in terms of the hotness and lifetime.

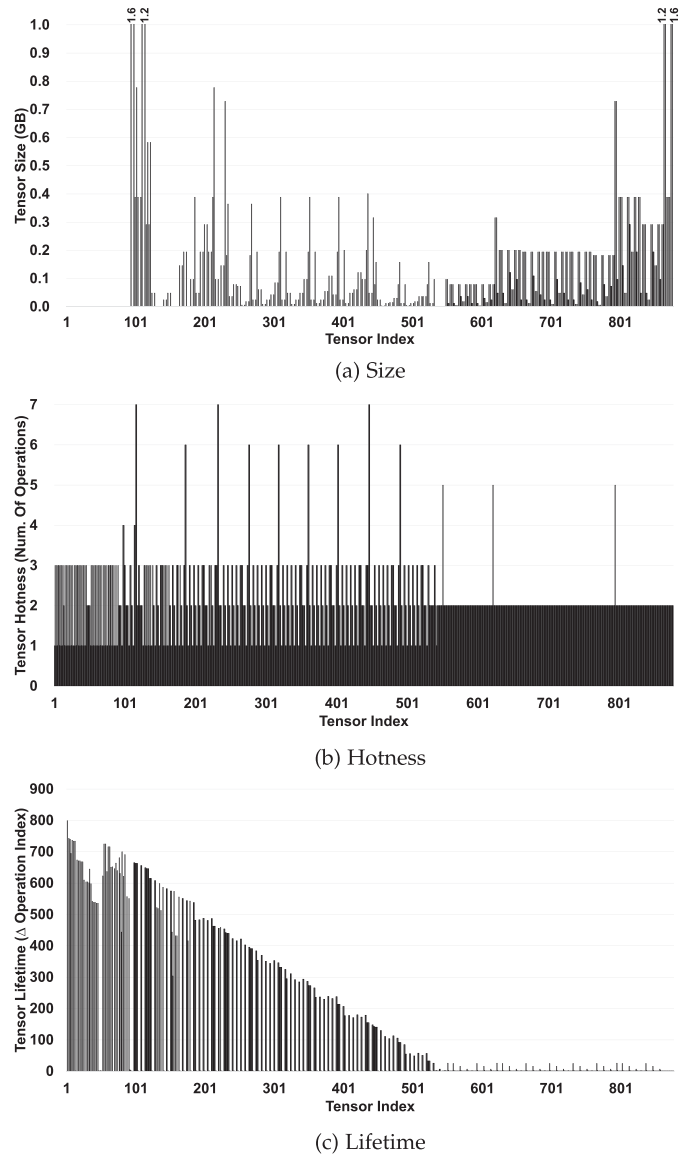


Fig. 6. Tensor characteristics of GN.

To accommodate this, the tensor combiner (virtually) combines all the tensors that share the same memory buffer into a single tensor and assigns the aggregated hotness and lifetime to the combined tensor.

Specifically, the tensor combiner sets the combined flag of tensors that share the same memory buffer to `true`, *except* for the tensor with the earliest allocation time among the tensors. The tensor combiner sets the combined flag of the tensor with the earliest allocation time among the tensors that share the same memory buffer to `false`. The tensor manager, which will be subsequently discussed in Section 5.4, only considers tensors whose combined flag is set to `false` to ensure that it places and migrates tensors with unique memory buffers.

The tensor combiner then aggregates all of the hotness and lifetime data across tensors that share the same memory buffer. The tensor combiner then assigns the aggregated hotness and lifetime to the tensor with the earliest allocation time (i.e., with its combined flag set to `false`) among the tensors. The aggregated hotness is computed as the sum of the hotness across the tensors. The aggregated allocation (or

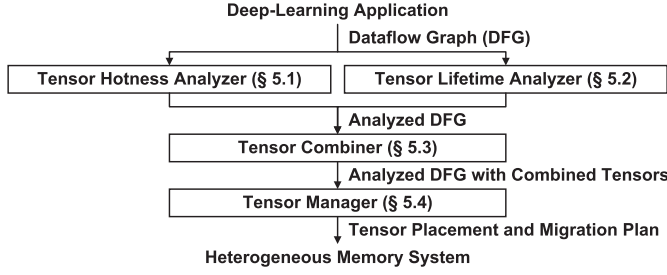


Fig. 7. Overall architecture of HALO.

deallocation) time is determined as the earliest (or latest) of all the allocation (or deallocation) times among the tensors.

5.4 Tensor Manager

5.4.1 Overview

The tensor manager of HALO constructs a plan for each tensor to dynamically place and migrate it across the heterogeneous memory nodes. Specifically, for each tensor, the tensor manager determines on which heterogeneous memory node it must be placed and which tensor (if any) must be migrated to the other heterogeneous memory node to secure sufficient memory space for the newly-allocated tensor.

Let s_t , h_t and B_P be the size of the tensor t , the hotness of the tensor t , and the effective bandwidth provided by the memory allocation policy P , where $P \in \{\text{LBM-Only}, \text{interleave}, \text{HBM-Only}\}$.⁵ We define the value ($v_{t,P}$) of the tensor t with the memory allocation policy P using Equation (1). Essentially, $v_{t,P}$ indicates the reduction in the data transfer time if the tensor t was allocated with the memory allocation policy P instead of the LBM-Only policy.

$$v_{t,P} = \begin{cases} \left(\frac{s_t}{B_{LBM}} - \frac{s_t}{B_{HBM}} \right) \cdot h_t & \text{if } P = \text{HBM-Only} \\ \left(\frac{s_t}{B_{LBM}} - \frac{s_t}{B_{interleave}} \right) \cdot h_t & \text{if } P = \text{interleave} \\ 0 & \text{if } P = \text{LBM-Only} \end{cases} \quad (1)$$

We also define the weight ($w_{t,P}$) of the tensor t with the memory allocation policy P using Equation (2). In essence, $w_{t,P}$ indicates the amount of HBM that would be consumed if the tensor t was allocated with the memory allocation policy P .

$$w_{t,P} = \begin{cases} s_t & \text{if } P = \text{HBM-Only} \\ \frac{s_t}{2} & \text{if } P = \text{interleave} \\ 0 & \text{if } P = \text{LBM-Only} \end{cases} \quad (2)$$

The tensor placement problem itself is a variant of the knapsack problem with time intervals, which is NP-hard [10]. Thus, we propose an efficient approximate algorithm that addresses the tensor placement and migration problem. Algorithm 1 shows the top-level function of the proposed algorithm, which mainly comprises two phases.

5.4.2 Phase 1: Initial Tensor Placement

During the first phase, the tensor manager aims to find an efficient allocation plan for all the tensors without considering

the migration (Line 4 in Algorithm 1). Algorithm 2 shows the top-level function for the first phase of the tensor manager. The first phase is based on a greedy algorithm in that it considers all the tensors in the order of their value per weight (Line 3) and makes a decision to gain the maximum value from each of the considered tensors.

Algorithm 1. The PlaceAndMigrateTensors Function

```

1: tensorList ← initTensorList(LBM-Only)
2: utilization ← initArray(numOperations, 0)
3: procedure PLACEANDMIGRATETENSORS
4:   generateTensorPlacementPlan()
5:   generateTensorPlacementAndMigrationPlan()
6: end procedure
  
```

Algorithm 2. The GenerateTensorPlacementPlan Function

```

1: procedure GENERATETENSORPLACEMENTPLAN
2:   slices ← generateSlices()
3:   sortedSliceList ← sort(slices, valPerWeight, DEC)
4:   for  $sl$  in sortedSliceList
5:     if isFeasible( $sl$ ) = true
6:       if  $sl.id = \text{FIRST}$ 
7:         setAllocPolicy( $sl.t$ , interleave)
8:       else  $\triangleright sl.id = \text{SECOND}$ 
9:         setAllocPolicy( $sl.t$ , HBM-Only)
10:      end if
11:      updateUtil( $sl.t.size/2$ ,  $sl.t.start$ ,  $sl.t.end$ )
12:    end if
13:  end for
14: end procedure
  
```

Since the value and weight of each tensor vary depending on which memory allocation policy (e.g., HBM-Only and interleave) is used to allocate the tensor, we introduce a concept called *tensor slices* (Line 2 in Algorithm 2). We assume that each tensor consists of two (virtual) tensor slices.

The first slice of a tensor represents the value (i.e., $v_{t,interleave}$) and weight (i.e., $\frac{s_t}{2}$) associated with the tensor when the allocation policy for the tensor is set to the interleave policy. The second slice of the tensor represents the additional value (i.e., $v_{t,HBM-Only} - v_{t,interleave}$) and weight (i.e., $\frac{s_t}{2}$) associated with the tensor if the allocation policy for the tensor is upgraded from the interleave policy to the HBM-Only policy. Note that the value per weight of the first tensor slice is always higher than the second slice.

The tensor manager visits each tensor slice in the (decreasing) order of the value per weight (Lines 2–13 in Algorithm 2). For each tensor slice, the tensor manager invokes (Line 5) the *isFeasible* function in Algorithm 3 to check if it can be allocated to HBM without exceeding the HBM capacity.

If the tensor slice can be allocated to HBM, the tensor manager checks whether it is the first slice of the associated tensor. If it is the first slice, the tensor manager sets the memory allocation policy of the associated tensor to the interleave policy (Lines 6–7 in Algorithm 2). Otherwise, the tensor manager upgrades the memory allocation policy of the associated tensor to the HBM-Only policy (Lines 8–9). Finally, the tensor manager accordingly updates the HBM usage information (Line 11) by invoking the *updateUtil* function in Algorithm 4 and proceeds with the second

5. On the server system evaluated in this work, $B_{LBM} = 80 \text{ GB/s}$, $B_{HBM} = 480 \text{ GB/s}$, and $B_{interleave} = 2 \cdot \min(B_{LBM}, B_{HBM}) = 160 \text{ GB/s}$.

phase of the tensor manager by invoking (Line 5 in Algorithm 1) the `generateTensorPlacementAndMigrationPlan` function in Algorithm 5.

Algorithm 3. The `IsFeasible` Function

```

1: procedure ISFEASIBLE(slice)
2:   feasibility  $\leftarrow$  true
3:   for  $i$  in [slice.start, slice.end]
4:     if slice.size > HBM.size – utilization[ $i$ ]
5:       feasibility  $\leftarrow$  false
6:       break
7:     end if
8:   end for
9:   return feasibility
10: end procedure

```

Algorithm 4. The `UpdateUtil` Function

```

1: procedure UPDATEUTIL(size, start, end)
2:   for  $i$  in [start, end]
3:     utilization[ $i$ ]  $\leftarrow$  utilization[ $i$ ] + size
4:   end for
5: end procedure

```

Algorithm 5. The `GenerateTensorPlacementAndMigrationPlan` Function

```

1: procedure GENERATE_TENSOR_PLACEMENT_AND_MIGRATION_PLAN
2:   sortedTensorList  $\leftarrow$  sort(tensorList, start, INC)
3:   for  $t$  in sortedTensorList
4:     plan1  $\leftarrow$  getCandidatePlan( $t$ , interleave)
5:     plan2  $\leftarrow$  getCandidatePlan( $t$ , HBM-Only)
6:     planbest  $\leftarrow$  maxProfitPlan(plan1, plan2)
7:     if planbest.profit > 0
8:       setAllocPolicy( $t$ , planbest.allocPolicy)
9:       if planbest.migrationFlag = true
10:        setVictim( $t$ , planbest.victim)
11:        setEvictPolicy( $t$ , planbest.evictPolicy)
12:       end if
13:       updateUtil(planbest.sizeChange,  $t$ .start,  $t$ .end)
14:     end if
15:   end for
16: end procedure

```

5.4.3 Phase 2: Refined Tensor Placement and Migration

During the second phase of the tensor manager, it refines the tensor placement plan produced during the first phase. Specifically, when the available space in HBM is insufficient to allocate a new tensor, placement of the newly allocated tensor in HBM during the first phase is disallowed. However, if there are currently some tensors in HBM that have already been accessed by most of their consumers, their values have already been significantly depreciated. Such tensors are good candidates for eviction to secure the sufficient space in HBM for a new tensor with a high value in order to increase the overall value.

To this end, the tensor manager visits each tensor in the (increasing) order of the allocation time in the sorted list (Lines 3–15 in Algorithm 5). At the allocation time of each

tensor, the tensor manager reevaluates the current values of all the tensors that have already been allocated in HBM and selects the victim tensor that maximizes the profit (Lines 4–6). Specifically, the profit is defined as the value of the newly-allocated tensor minus the remaining value and migration (i.e., from HBM to LBM) cost of the victim tensor. If the value of the newly allocated tensor is higher than the cost of the victim tensor, the tensor manager evicts the victim tensor and places the newly allocated tensor in HBM (Lines 7–12) because it increases the overall value. Finally, the tensor manager accordingly updates the HBM usage information (Line 13).

5.5 Discussion

5.5.1 Asymptotic Time Complexity Analysis

The time complexity of the first phase of the tensor manager is $O(N_T \cdot \log N_T + N_T \cdot N_O)$ because the most time-consuming part consists of Line 3 (i.e., $O(N_T \cdot \log N_T)$) or Lines 4 and 11 (i.e., $O(N_T \cdot N_O)$) in Algorithm 2.⁶ The time complexity of the second phase of the tensor manager is $O(N_T^2 + N_T \cdot N_O)$ because the most time-consuming part is Lines 3 and 4 (i.e., $O(N_T^2)$) or Lines 3 and 13 (i.e., $O(N_T \cdot N_O)$) in Algorithm 5. Therefore, the overall time complexity of the tensor manager is $O(N_T^2 + N_T \cdot N_O)$. As quantified in Section 6.2, HALO incurs small performance overheads.

5.5.2 Implementation

We have implemented HALO by extending the TensorFlow 1.4.0. Specifically, we have added or modified 39 files and 3,726 lines of code (C++ and Python) in TensorFlow to implement HALO. Most of the code for HALO is DL framework-independent as it implements high-level optimizations based on the dataflow graph and can be widely applied to other DL frameworks (e.g., PyTorch [29]).

5.5.3 Memory Allocation Ratios

For clarity and conciseness, we have discussed HALO with the assumption that the underlying heterogeneous memory system provides the three memory allocation ratios (i.e., LBM-Only, HBM-Only, and interleave). However, HALO can be readily extended to support heterogeneous memory systems with four or more memory allocation ratios.

Specifically, if the underlying heterogeneous memory system provides N (non-zero) memory allocation ratios, HALO can be extended to logically decompose each tensor into $N - 1$ (virtual) slices. HALO then sorts the slices of all the tensors in the order of the value per weight and places each slice in the same order if there is sufficient space for the slice. If M slices ($M \leq N - 1$) of a tensor are allowed to be placed in HBM, the tensor can be allocated across the heterogeneous memory nodes in a way to employ the $(N - M)$ -th highest memory bandwidth that can be provided by the underlying heterogeneous memory system.

6 EVALUATION

This section quantifies the effectiveness of HALO. Specifically, we aim to investigate the following – (1) the

6. N_T and N_O denote the total number of the tensors and the operations in the target DL application, respectively.

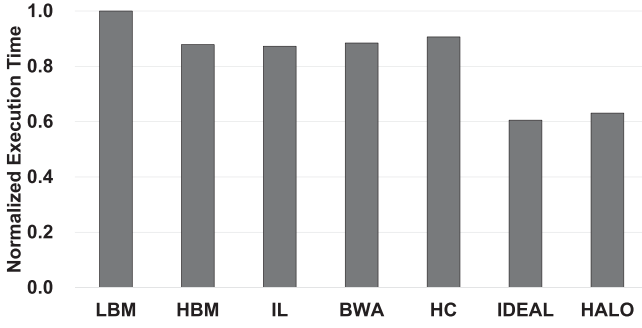


Fig. 8. Overall performance results.

performance and energy consumption of HALO and various memory management policies, (2) the performance overheads of HALO, (3) the performance sensitivity of HALO to the application working-set size, and (4) the performance impact of the individual optimization techniques of HALO.

6.1 Performance and Energy Results

To quantify the performance and energy consumption impact of HALO and various software- and hardware-based memory management policies, we run each of the nine deep-learning (DL) applications summarized in Table 2 with the following seven versions – (1) LBM-Preferred (LBM), (2) HBM-Preferred (HBM), (3) interleave (IL), (4) bandwidth aware (BWA), (5) hardware cache (HC), (6) ideal case with infinite HBM (IDEAL), and (7) HALO. With regard to the first three versions, we employ the corresponding memory policies in Linux with the Flat mode on the Xeon Phi KNL architecture.

For the BWA version, we employ a bandwidth-aware memory management policy similar to the ones proposed in [9], [44] with the Flat mode. When allocating the memory buffer for each tensor, the bandwidth-aware memory management policy aims to employ pages from the HBM and LBM nodes with the optimal allocation ratios of $\frac{B_{HBM}}{B_{HBM}+B_{LBM}}$ and $\frac{B_{LBM}}{B_{HBM}+B_{LBM}}$, which are mathematically proven to maximize the effective bandwidth across the heterogeneous memory nodes [9], [44]. If there is insufficient space in HBM required for a tensor, the bandwidth-aware memory management policy uses all the available space in HBM that can accommodate a portion of the tensor and allocates the remaining portion of the tensor to LBM.

Regarding the HC version, we employ the interleave memory policy in Linux with the Cache mode on the Xeon

Phi KNL architecture. Note that, in the Cache mode, HBM is fully managed by hardware as a last-level cache (i.e., transparent to the OS).

For the IDEAL version, we estimate the ideal performance that can be potentially achieved on an imaginary system that has HBM with infinite capacity. To this end, we execute each layer of the target DL application by allocating all the tensors of the layer to HBM and add the execution times of all the layers. Finally, regarding the HALO version, we employ HALO with the Flat mode. Unless specified otherwise, we use the medium batch size for each application.

We first analyze the overall performance results of HALO and the other memory management policies. Fig. 8 shows the execution time of each version, which is the average across all the evaluated applications. Each bar denotes the execution time of each version, normalized to the LBM-Preferred version.

We observe that HALO significantly outperforms the other memory management policies. Specifically, HALO achieves 36.9, 28.2, 27.7, 28.7, and 30.4 percent higher performance than the LBM-Preferred, HBM-Preferred, IL, BWA, and HC versions across the evaluated applications on average (i.e., geometric mean).

In addition, HALO achieves the performance comparable to the IDEAL version. Specifically, HALO exhibits 4.2 percent lower performance on average than the IDEAL version, which assumes an imaginary system equipped with infinite HBM. In summary, the overall performance results clearly demonstrate the effectiveness of HALO.

To gain a deeper insight into the overall performance trends, Fig. 9 shows a breakdown of the execution time of each version of the applications, normalized to the LBM-Preferred version. Each bar consists of the times spent for executing the most time-consuming operations.

We observe that HALO achieves significantly higher performance than the other memory management policies across all the evaluated DL applications. In particular, HALO provides larger performance gains over the other memory management policies with the applications (e.g., DN and RNv2) that employ non-linear neural networks. With sophisticated neural networks, the diversity of the tensors increases in terms of the hotness and lifetime. Since HALO effectively places and migrates tensors based on their characteristics, it delivers larger performance gains with the applications with non-linear networks.

We also observe that HALO significantly reduces the execution times of bandwidth-sensitive operations (e.g.,

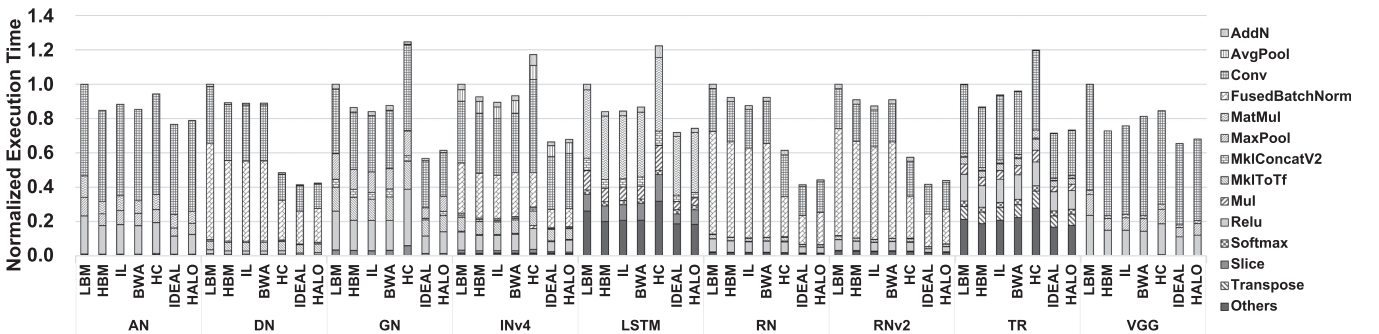


Fig. 9. Execution breakdowns with HALO and various memory management policies.

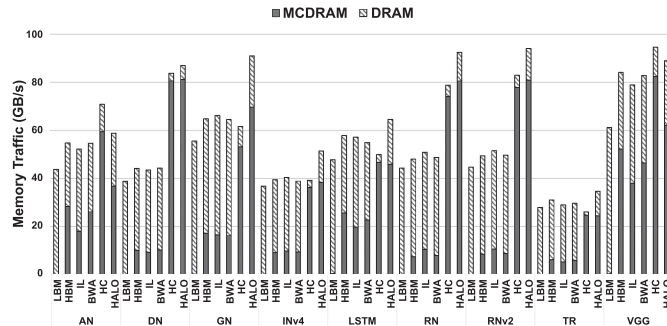


Fig. 10. Memory traffic.

FusedBatchNorm, MaxPool, Relu) by effectively placing the tensors in a hotness- and lifetime-aware manner. In contrast, since the other software-based memory management policies place less critical tensors to HBM owing to the lack of capability of utilizing the hotness and lifetime characteristics on the tensors, they are significantly outperformed by HALO.

In addition to the aforementioned reasons, the bandwidth-aware memory management policy achieves lower performance than HALO because the working-set size of the evaluated applications is larger than the capacity of HBM, making it impossible to maintain the optimal memory allocation ratio when allocating tensors across the HBM and LBM nodes. Further, since the bandwidth of LBM is significantly lower than that of HBM (i.e., $B_{LBM} = 80$ GB/s, $B_{HBM} = 480$ GB/s), it has relatively small impact on the overall performance.

The HC version achieves lower performance than HALO because it allocates the data in a hotness- and lifetime-oblivious manner. Further, since HBM is managed as a hardware cache with the HC version, it incurs extra traffic for the data to be cached and for secondary operations [14], degrading its overall performance.

The IDEAL version slightly outperforms HALO as it places all the data in HBM with infinite capacity. Nevertheless, HALO, which employs HBM with finite capacity, achieves the performance comparable (i.e., 4.2 percent lower on average) to that of the IDEAL version through hotness- and lifetime-aware tensor placement and migration. This data trend clearly demonstrates the effectiveness of HALO.

As shown in Fig. 10, the performance differences of HALO and the other memory management policies can be also explained in terms of memory traffic. HALO dynamically places and migrates tensors in a hotness- and lifetime-aware manner, achieving high memory traffic. In contrast, since the software-based memory management policies (i.e., LBM-Preferred, HBM-Preferred, IL, and BWA) place tensors without considering the hotness and lifetime characteristics of the tensors, they incur low memory traffic, achieving significantly lower performance than HALO.

The HC version of some applications incurs relatively high memory traffic (e.g., AN, VGG). This is mainly due to the extra memory traffic caused by the data placed in or evicted to LBM in a hotness- and lifetime-oblivious manner and the secondary operations for cache management [14]. The increased memory traffic incurs significant performance overheads. In contrast, HALO effectively utilizes the off-chip memory bandwidth by placing and migrating the

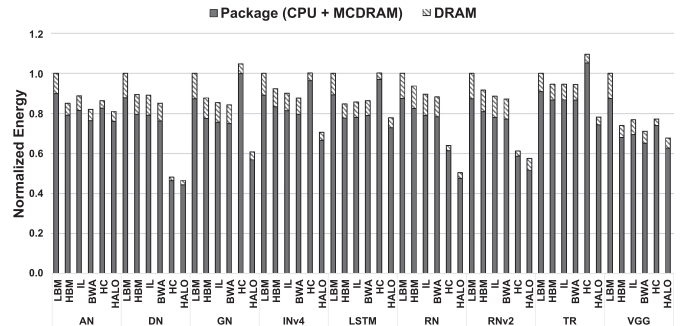


Fig. 11. Energy consumption breakdowns.

tensors in a hotness- and lifetime-aware manner, significantly outperforming the HC version.

We also analyze the overall energy consumption results of HALO and the other memory management policies. Fig. 11 shows a breakdown of the energy consumption of each version of the applications. Each bar denotes the energy consumption normalized to the LBM-Preferred version and consists of the energy consumption of the package (i.e., CPU and MCDRAM) and DRAM.⁷

First, similarly to the performance results, HALO consumes significantly less energy than the other memory management policies, demonstrating the effectiveness of HALO. Specifically, HALO consumes 34.6, 25.8, 24.9, 23.3, and 24.0 percent less energy than the LBM-Preferred, HBM-Preferred, IL, BWA, and HC versions, respectively. Since one of the major factors that determine the overall energy consumption of each version is its execution time, the energy consumption data trends are similar to the performance data trends.

Second, interestingly, the DRAM energy consumption accounts for a smaller portion of the total energy consumption across all the DL applications with HALO than the software-based memory management policies (i.e., LBM-Preferred, HBM-Preferred, IL, and BWA). Since HALO reduces the traffic to LBM (i.e., DRAM) by allocating frequently-accessed tensors to HBM, it reduces the DRAM energy consumption. In contrast, as the software-based memory management policies generate more traffic to LBM by allocating tensors in a hotness- and lifetime-oblivious manner, they exhibit higher DRAM energy consumption.

The HC version exhibits relatively low DRAM energy consumption because the data is frequently copied from LBM to HBM and accessed from HBM, which is managed as a hardware cache. However, since the HC version achieves lower performance than HALO, its overall energy consumption is higher than HALO.

6.2 Performance Overheads

We investigate the performance overheads of the tensor placement and migration planning of HALO. Fig. 12 shows the percentage of the time spent to generate the tensor

7. Since the evaluated system lacks the capability of separately monitoring the energy consumption of the CPU and MCDRAM in the package, we present the combined energy consumption (i.e., the package energy consumption) data for the CPU and MCDRAM. Considering the DRAM energy consumption data trends and the data-intensive nature of the DL applications, we conjecture that the MCDRAM energy consumption would account for a considerable portion of the total package energy consumption.

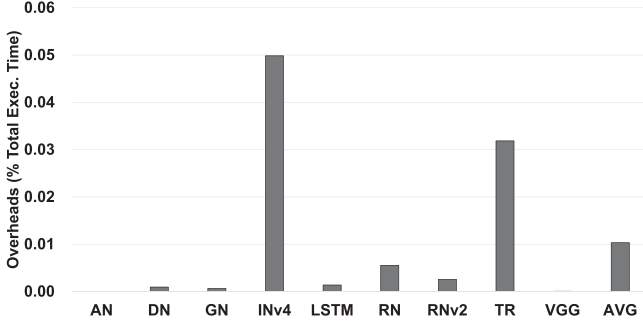


Fig. 12. Performance overheads of HALO.

placement and migration plan out of the total execution time for each application.

First, the performance overheads of HALO are small across all the evaluated applications. For instance, the average performance overhead of HALO across the evaluated applications is 0.010 percent of the total execution time, which is small. Because HALO employs the efficient algorithm to generate the tensor placement and migration plan, it incurs small performance overheads. Second, some applications (e.g., INv4) incur larger (but still small) overheads than the other applications. Since their dataflow graphs in terms of the operation and tensor counts are larger than those of the other applications, HALO spends a longer time to find the efficient placement and migration plan for the tensors with such applications. Nevertheless, the overall performance overheads of HALO are small across all the evaluated applications, demonstrating its efficiency.

6.3 Performance Sensitivity

We investigate the performance sensitivity of HALO to the application working-set size. Fig. 13 shows the execution time ratio of the HALO version of each application to the HBM-Preferred version with different batch sizes (i.e., small, medium, large).⁸ Note that the application working-set size increases as the batch size increases. We observe the following data trends.

First, HALO consistently outperforms the HBM-Preferred policy across various working-set sizes, demonstrating its effectiveness. Second, the execution time ratio of HALO to the HBM-Preferred policy tends to decrease as the application working-set size increases. With a larger working-set size of the target DL application, only a small portion of the tensors can be accommodated by HBM. Since the HBM-Preferred policy places the tensors without considering their hotness and lifetime characteristics, it frequently places inefficient tensors in HBM, resulting in low performance. In contrast, HALO dynamically places and migrates the tensors in a hotness- and lifetime-aware manner, achieving significantly higher performance than the HBM-Preferred policy.

Overall, our quantitative evaluation demonstrates the effectiveness of HALO in that it achieves significantly higher performance and lower energy consumption than the representative five software- and hardware-based memory

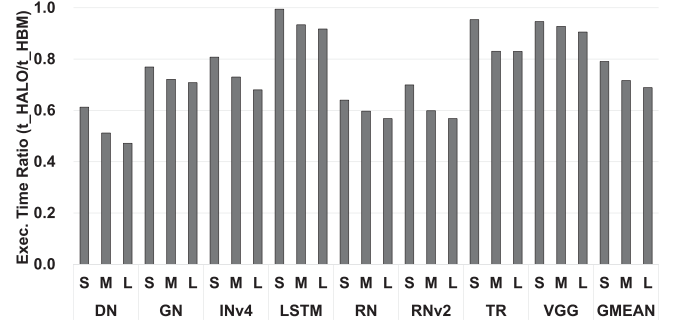


Fig. 13. Sensitivity to the application working-set size.

management policies, delivers the performance comparable to the ideal case with infinite HBM, incurs small performance overheads, and exhibits high performance across a wide range of the application working-set sizes when executing various DL applications.

6.4 Impact of the Optimization Techniques

Finally, we quantify the performance impact of the individual optimization techniques of HALO. To this end, we create two intermediate versions of HALO that only perform hotness-aware tensor placement and hotness- and lifetime-aware tensor placement, respectively. Fig. 14 shows the ratio of the average (i.e., geometric mean across the evaluated applications) execution time of the two intermediate versions of HALO and the full version (i.e., hotness- and lifetime-aware tensor placement and migration) of HALO to the intermediate version, which only performs hotness-aware tensor placement.

We observe that each optimization technique constructively composes with the other optimization techniques in that the performance of HALO continues to improve as more techniques are applied. Further, each optimization technique considerably contributes to improving the overall performance, demonstrating the effectiveness of HALO.

7 RELATED WORK

Prior works have extensively investigated system software techniques to improve the performance and scalability of the parallel and distributed deep-learning (DL) systems [7], [11], [13], [20], [21], [26], [28], [31], [40], [47]. While impactful, most prior works propose optimization techniques on homogeneous memory systems [7], [13], [20], [21], [28], [47]. Our work significantly differs in that it investigates the hotness- and lifetime-aware tensor placement and

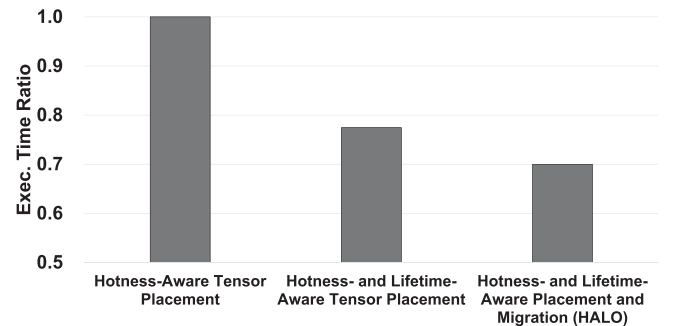


Fig. 14. Impact of the optimization techniques.

8. AN is excluded because it fails to run with the large batch size on the evaluated server system.

migration techniques on heterogeneous memory systems, where data can be directly placed, accessed, and migrated across the heterogeneous memory nodes in a fully parallel manner.

Some of the prior works are closely related to ours [11], [26], [31], [40]. Based on the memory usage pattern of each layer in the target application, the techniques presented in [11], [26], [31], [40] dynamically transfer the data between the CPU and GPU memory [11], [31], [40] or the memory modules attached to the device-side interconnection network (e.g., NVIDIA NVLink) and GPU memory [26] to ensure that the GPU memory usage of the target application does not exceed the capacity of the GPU memory during the entire execution time of the target application. While insightful, those works lack the consideration of the hotness of tensors because the data in the CPU host memory [11], [31], [40] or the memory modules within the device-side interconnect [26] cannot be directly accessed by the computation units in the GPU. Our work significantly differs in the sense that HALO dynamically places and migrates the tensors based on their hotness and lifetime characteristics to significantly improve the performance of the target DL applications on heterogeneous memory systems on which the computation units can directly and simultaneously access all the heterogeneous memory nodes.

Prior works have investigated the design and implementation of hardware accelerators for high-performance and low-power DL [12], [19], [36]. While effective, the prior works primarily focus on the design and implementation of computation units that are customized for efficiently executing the mathematical operations (e.g., convolution) that are frequently employed by DL applications and lack the consideration of the performance issues on heterogeneous memory systems. We believe that heterogeneity-aware memory management techniques such as HALO will become more crucial with the widespread use of the hardware accelerators as the data transfer time is expected to account for a larger portion of the total execution time of DL applications as the hardware accelerators effectively reduce the time spent for computations.

Prior works have extensively explored architectural [9], [14], [18], [22], [39], [43], [45] and system software [33], [41], [42], [44] techniques for heterogeneous memory systems. Some of the prior works [9], [22], [33], [39], [41], [42], [44] investigate the placement [9], [22], [41], [44] and/or migration [33], [39], [42], [44] policies for heterogeneous memory systems that employ the flat memory organization. The other prior works [14], [18], [43], [45] explore the cache management (e.g., placement, replacement, bypassing) policies [18], [43] and the techniques to reduce the traffic between memory nodes [14], [45] for heterogeneous memory systems with the tiered memory organization.

While insightful, the prior works lack the consideration of application-level information, which is readily available from the dataflow graph of the target DL application. In contrast, HALO robustly extracts the application-level information (e.g., hotness, lifetime) from the target DL application and effectively places and migrates tensors across the heterogeneous memory nodes based on the application-level information.

8 CONCLUSION

In this work, we investigate the characteristics of various deep-learning (DL) applications on a real heterogeneous memory system. Guided by the characterization results, we propose HALO, hotness- and lifetime-aware data placement and migration for high-performance DL on heterogeneous memory systems. HALO dynamically analyzes the hotness and lifetime characteristics of the tensors of the target DL application and places the tensors across the heterogeneous memory nodes in a hotness- and lifetime-conscious manner. Our experimental results demonstrate the effectiveness of HALO in that it significantly outperforms various memory management policies supported by the underlying system software and hardware, achieves the performance comparable to the ideal case with infinite HBM, incurs small performance overheads, and delivers high performance across a wide range of the application working-set sizes. As future work, we plan to investigate efficient system software support for high-performance DL on highly heterogeneous computing systems that comprise heterogeneous computing (e.g., CPU, GPU, FPGA, NPU) and memory devices.

ACKNOWLEDGMENTS

This research was partly supported by the National Research Foundation of Korea (NRF-2016M3C4A7952587, PF Class Heterogeneous High Performance Computer Development), Basic Science Research Program through the National Research Foundation of Korea (NRF-2018R1C1B6005961), and Institute of Information & Communications Technology Planning & Evaluation (IITP) Grant funded by the Korea Government (MSIT) (No.1711080972, Neuromorphic Computing Software Platform for Artificial Intelligence Systems).

REFERENCES

- [1] "Convnet benchmarks," 2016. [Online]. Available: <https://github.com/soumith/convnet-benchmarks>
- [2] "Intel optimized tensorflow wheel now available," 2017. [Online]. Available: <https://software.intel.com/en-us/articles/intel-optimized-tensorflow-wheel-now-available>
- [3] "Introducing dnn primitives in Intel® math kernel library," 2017. [Online]. Available: <https://software.intel.com/en-us/articles/introducing-dnn-primitives-in-intel-mkl>
- [4] "Tensor2tensor - transformer," 2017. [Online]. Available: <https://github.com/tbd-ai/tbd-suite/tree/master/MachineTranslation-Transformer/Tensorflow>
- [5] "Tensorflow benchmarks," 2017. [Online]. Available: <https://github.com/tensorflow/benchmarks>
- [6] "Tensorflow optimizations on modern intel® architecture," 2017. [Online]. Available: <https://software.intel.com/en-us/articles/tensorflow-optimizations-on-modern-intel-architecture>
- [7] M. Abadi et al., "Tensorflow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Syst. Design Implementation*, 2016, pp. 265–283. [Online]. Available: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>
- [8] R. Adolf, S. Rama, B. Reagen, G. y. Wei, and D. Brooks, "Fathom: reference workloads for modern deep learning methods," in *Proc. IEEE Int. Symp. Workload Characterization*, Sep. 2016, pp. 1–10.
- [9] N. Agarwal, D. Nellans, M. Stephenson, M. O'Connor, and S. W. Keckler, "Page placement strategies for GPUs within heterogeneous memory systems," in *Proc. 20th Int. Conf. Architectural Support Program. Languages Operating Syst.*, 2015, pp. 607–618. [Online]. Available: <http://doi.acm.org/10.1145/2694344.2694381>
- [10] M. Bartlett, A. M. Frisch, Y. Hamadi, I. Miguel, S. A. Tarim, and C. Unsworth, "The temporal knapsack problem and its solution," in *Proc. 2nd Int. Conf. Integration AI OR Techn. Constraint Program. Combinatorial Optimization Problems*, 2005, pp. 34–48. [Online]. Available: http://dx.doi.org/10.1007/11493853_5

- [11] X. Chen, D. Z. Chen, and X. S. Hu, "moDNN: Memory optimal DNN training on GPUs," in *Proc. Design Automation Test Eur. Conf. Exhib.*, Mar. 2018, pp. 13–18.
- [12] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proc. 43rd Int. Symp. Comput. Architecture*, 2016, pp. 367–379. [Online]. Available: <https://doi.org/10.1109/ISCA.2016.40>
- [13] T. Chilimbi, Y. Suzue, J. Apacible, and K. Kalyanaraman, "Project adam: Building an efficient and scalable deep learning training system," in *Proc. 11th USENIX Conf. Operating Syst. Design Implementation*, 2014, pp. 571–582. [Online]. Available: <http://dl.acm.org/citation.cfm?id=2685048.2685094>
- [14] C. Chou, A. Jaleel, and M. K. Qureshi, "Bear: Techniques for mitigating bandwidth bloat in gigascale dram caches," in *Proc. 42nd Annu. Int. Symp. Comput. Architecture*, 2015, pp. 198–210. [Online]. Available: <http://doi.acm.org/10.1145/2749469.2750387>
- [15] A. Colin, G. Harvey, B. Lucia, and A. P. Sample, "An energy-interference-free hardware-software debugger for intermittent energy-harvesting systems," in *Proc. 21st Int. Conf. Architectural Support Program. Languages Operating Syst.*, 2016, pp. 577–589. [Online]. Available: <http://doi.acm.org/10.1145/2872362.2872409>
- [16] S. R. Dulloor et al., "System software for persistent memory," in *Proc. 9th Eur. Conf. Comput. Syst.*, 2014, pp. 15:1–15:15. [Online]. Available: <http://doi.acm.org/10.1145/2592798.2592814>
- [17] F. Gaud, B. Lepers, J. Decouchant, J. Funston, A. Fedorova, and V. Quéma, "Large pages may be harmful on NUMA systems," in *Proc. USENIX Annu. Tech. Conf.*, 2014, pp. 231–242. [Online]. Available: <http://dl.acm.org/citation.cfm?id=2643634.2643659>
- [18] J. Gaur, M. Chaudhuri, P. Ramachandran, and S. Subramoney, "Near-optimal cache partitioning for memory hierarchies with multiple heterogeneous bandwidth sources," in *Proc. IEEE Int. Symp. High Performance Comput. Architecture*, Feb. 2017, pp. 13–24.
- [19] S. Han et al., "Eie: Efficient inference engine on compressed deep neural network," in *Proc. 43rd Int. Symp. Comput. Architecture*, 2016, pp. 243–254. [Online]. Available: <https://doi.org/10.1109/ISCA.2016.30>
- [20] A. Jain, A. Phanishayee, J. Mars, L. Tang, and G. Pekhimenko, "Gist: Efficient data encoding for deep neural network training," in *Proc. ACM/IEEE 45th Annu. Int. Symp. Comput. Architecture*, Jun. 2018, pp. 776–789.
- [21] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell, "Caffe: Convolutional architecture for fast feature embedding," in *Proc. 22nd ACM Int. Conf. Multimedia*, 2014, pp. 675–678. [Online]. Available: <http://doi.acm.org/10.1145/2647868.2654889>
- [22] K. Y. Kim and W. Baek, "BLPP: Improving the performance of GPGPUs with heterogeneous memory through bandwidth- and latency-aware page placement," in *Proc. IEEE 36th Int. Conf. Comput. Design*, Oct. 2018, pp. 358–365.
- [23] A. Kolli, S. Pelley, A. Saidi, P. M. Chen, and T. F. Wenisch, "High-performance transactions for persistent memories," in *Proc. 21st Int. Conf. Architectural Support Program. Languages Operating Syst.*, 2016, pp. 399–411. [Online]. Available: <http://doi.acm.org/10.1145/2872362.2872381>
- [24] A. Krizhevsky, "Learning multiple layers of features from tiny images," 2009.
- [25] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proc. 25th Int. Conf. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105. [Online]. Available: <http://dl.acm.org/citation.cfm?id=2999134.2999257>
- [26] Y. Kwon and M. Rhu, "Beyond the memory wall: A case for memory-centric hpc system for deep learning," in *Proc. 51st Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2018, pp. 148–161. [Online]. Available: <https://doi.org/10.1109/MICRO.2018.00021>
- [27] M. P. Marcus, B. Santorini, M. A. Marcinkiewicz, and A. Taylor, "Treebank-3," 1999. [Online]. Available: <https://catalog.ldc.upenn.edu/Ldc99t42>
- [28] A. Mirhoseini et al., "Device placement optimization with reinforcement learning," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 2430–2439. [Online]. Available: <http://proceedings.mlr.press/v70/mirhoseini17a.html>
- [29] A. Paszke et al., "Automatic differentiation in pytorch," in *Proc. Conf. Neural Inf. Process. Syst.*, 2017.
- [30] S. Ramos and T. Hoefler, "Capability models for manycore memory systems: A case-study with xeon phi KNL," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, May 2017, pp. 297–306.
- [31] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfiqar, and S. W. Keckler, "vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2016, pp. 18:1–18:13. [Online]. Available: <http://dl.acm.org/citation.cfm?id=3195638.3195660>
- [32] O. Russakovsky et al., "Imagenet large scale visual recognition challenge," *Int. J. Comput. Vision*, vol. 115, no. 3, pp. 211–252, Dec. 2015. [Online]. Available: <http://dx.doi.org/10.1007/s11263-015-0816-y>
- [33] J. H. Ryoo, L. K. John, and A. Basu, "A case for granularity aware page migration," in *Proc. Int. Conf. Supercomputing*, 2018, pp. 352–362. [Online]. Available: <http://doi.acm.org/10.1145/3205289.3208064>
- [34] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *CoRR*, vol. abs/1409.1556, 2014. [Online]. Available: <http://arxiv.org/abs/1409.1556>
- [35] A. Sodani et al., "Knights landing: Second-generation intel xeon phi product," *IEEE Micro*, vol. 36, no. 2, pp. 34–46, Mar. 2016. [Online]. Available: <http://dx.doi.org/10.1109/MM.2016.25>
- [36] L. Song, Y. Wang, Y. Han, X. Zhao, B. Liu, and X. Li, "C-Brain: A deep learning accelerator that tames the diversity of CNNs through adaptive data-level parallelization," in *Proc. 53rd Annu. Design Automat. Conf.*, 2016, pp. 123:1–123:6. [Online]. Available: <http://doi.acm.org/10.1145/2897937.2897995>
- [37] C. Szegedy et al., "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recog.*, Jun. 2015, pp. 1–9.
- [38] A. Vaswani et al., "Attention is all you need," *CoRR*, vol. abs/1706.03762, 2017. [Online]. Available: <http://arxiv.org/abs/1706.03762>
- [39] H. Wang, J. Zhang, S. Shridhar, G. Park, M. Jung, and N. S. Kim, "DUANG: Fast and lightweight page migration in asymmetric memory systems," in *Proc. IEEE Int. Symp. High Performance Comput. Architecture*, Mar. 2016, pp. 481–493.
- [40] L. Wang et al., "Superneurons: Dynamic GPU memory management for training deep neural networks," in *Proc. 23rd ACM SIGPLAN Symp. Principles Practice Parallel Program.*, 2018, pp. 41–53. [Online]. Available: <http://doi.acm.org/10.1145/3178487.3178491>
- [41] S. Wen, L. Cherkasova, F. X. Lin, and X. Liu, "ProfDP: A lightweight profiler to guide data placement in heterogeneous memory systems," in *Proc. Int. Conf. Supercomputing*, 2018, pp. 263–273. [Online]. Available: <http://doi.acm.org/10.1145/3205289.3205320>
- [42] Z. Yan, D. Lustig, D. Nellans, and A. Bhattacharjee, "Nimble page management for tiered memory systems," in *Proc. 24th Int. Conf. Architectural Support Program. Languages Operating Syst.*, 2019, pp. 331–345. [Online]. Available: <http://doi.acm.org/10.1145/3297858.3304024>
- [43] V. Young, C. Chou, A. Jaleel, and M. K. Qureshi, "Accord: Enabling associativity for gigascale dram caches by coordinating way-install and way-prediction," in *Proc. 45th Annu. Int. Symp. Comput. Architecture*, 2018, pp. 328–339. [Online]. Available: <https://doi.org/10.1109/ISCA.2018.00036>
- [44] S. Yu, S. Park, and W. Baek, "Design and implementation of bandwidth-aware memory placement and migration policies for heterogeneous memory systems," in *Proc. Int. Conf. Supercomputing*, 2017, pp. 18:1–18:10. [Online]. Available: <http://doi.acm.org/10.1145/3079079.3079092>
- [45] X. Yu, C. J. Hughes, N. Satish, O. Mutlu, and S. Devadas, "Banshee: Bandwidth-efficient dram caching via software/hardware cooperation," in *Proc. 50th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2017, pp. 1–14. [Online]. Available: <http://doi.acm.org/10.1145/3123939.3124555>
- [46] W. Zaremba, I. Sutskever, and O. Vinyals, "Recurrent neural network regularization," *CoRR*, vol. abs/1409.2329, 2014. [Online]. Available: <http://arxiv.org/abs/1409.2329>
- [47] H. Zhang et al., "Poseidon: An efficient communication architecture for distributed deep learning on GPU clusters," in *Proc. USENIX Annu. Tech. Conf.*, 2017, pp. 181–193. [Online]. Available: <https://www.usenix.org/conference/atc17/technical-sessions/presentation/zhang>



Myeonggyun Han received the bachelor's degree from the Ulsan National Institute of Science and Technology, in 2018. He is working toward the integrated MS/PhD degrees in the Department of Computer Engineering, Ulsan National Institute of Science and Technology. His research interests include parallel and distributed computing and computer systems security.



Jihoon Hyun received the bachelor's degree from the Ulsan National Institute of Science and Technology, in 2016. He is working toward the MS degree in the Department of Computer Engineering, Ulsan National Institute of Science and Technology. His research interests include parallel and embedded computing.



Woongki Baek received the BS and MS degrees from Seoul National University, and the PhD degree in electrical engineering from Stanford University, in 2011. He is an associate professor with the School of Electrical and Computer Engineering, Ulsan National Institute of Science and Technology. Prior to joining UNIST, he worked for Microsoft as a Platform Software Engineer and for ETRI as a Senior Researcher. He was the recipient of the Stanford Graduate Fellowship and the Samsung Scholarship. His research interests include architecture, system software, and programming models for parallel computing. He is a member of the ACM and IEEE.



Seongbeom Park received the bachelor's degree from the Ulsan National Institute of Science and Technology, in 2015. He is working toward the integrated MS/PhD degree in the Department of Computer Engineering, Ulsan National Institute of Science and Technology. His research interests include parallel computing and garbage collection.

▷ **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**